Introduction to NumPy and Pandas

Topics Covered

- NumPy Arrays
- Mathematical Operations
- Pandas Series and DataFrames
- Data Cleaning, Sorting, and Filtering

What is NumPy?

- NumPy stands for Numerical Python.
- Used for numerical computing and data processing.
- Provides support for multidimensional arrays.
- Faster and more efficient than Python lists.

NumPy vs Python List

Difference Between NumPy Array and Python List

Feature	NumPy Array	Python List
Speed	Faster	Slower
Memory Usage	Less Memory	More Memory
Mathematical Operations	Directly Supported	Requires Loops
Data Type	Same Data Type	Different Data Types
Performance	High	Moderate

Example

```
import numpy as np

list_data = [1, 2, 3, 4]
array_data = np.array([1, 2, 3, 4])
```

Conclusion

- NumPy arrays are faster and more efficient.
- Python lists are more flexible but slower for numerical computations.

Creating NumPy Arrays

Example

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
```

Advantages

- Fast computation
- Less memory usage
- Easy mathematical operations

Mathematical Operations

Example

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b
a * b
```

Operations

- Addition
- Subtraction
- Multiplication
- Division

Statistical Functions

```
arr.mean()
arr.max()
arr.min()
arr.sum()
```

Common Functions

- mean()

- max()
- min()
- sum()
- std()

Introduction to Pandas

- Pandas is a Python library for data analysis.
- Used to organize and manipulate data.
- Main structures:
- Series
- DataFrame

Pandas Series

Example

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
```

Features

- One-dimensional data structure
- Similar to an Excel column

Pandas DataFrame

Example

```
data = {
"Name": ["Ali", "Sara", "Ahmed"],
"Marks": [85, 90, 78]
}

df = pd.DataFrame(data)
```

Features

- Rows and Columns
- Easy data analysis

Sorting Data

df.sort_values("Marks")

Purpose

- Arrange data in ascending order
- Arrange data in descending order

Filtering Data

df[df["Marks"] > 80]

Benefits

- Display specific records
- Quick data searching

Data Cleaning

Check Missing Values

df.isnull()

Remove Missing Values

df.dropna()

Fill Missing Values

df.fillna(0)

Importance

- Better data quality
- Accurate analysis

Plotting with Matplotlib

Topics Covered

- Introduction to Matplotlib
- Line Charts
- Bar Charts
- Pie Charts
- Data Visualization Basics

What is Matplotlib?

- Matplotlib is a Python library used for data visualization.
- It helps create graphs and charts easily.
- Widely used in Data Science and Data Analysis.
- Supports various types of plots.

Import Matplotlib

```
import matplotlib.pyplot as plt
```

Creating a Line Chart

Example

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.show()
```

Uses

- Show trends over time
- Compare continuous data

Customizing Line Charts

Example

```
plt.plot(x, y, color='blue', marker='o')
plt.title("Line Chart")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.show()
```

Features

- Titles
- Labels
- Colors
- Markers

Creating a Bar Chart

Example

```
categories = ["A", "B", "C"]
values = [10, 20, 15]

plt.bar(categories, values)
plt.show()
```

Uses

- Compare categories
- Display discrete data

Customizing Bar Charts

Example

```
plt.bar(categories, values, color='green')
plt.title("Bar Chart")
plt.xlabel("Categories")
```

```
plt.ylabel("Values")
plt.show()
```

Benefits

- Easy comparison
- Clear visualization

Creating a Pie Chart

Example

```
sizes = [40, 35, 25]
labels = ["A", "B", "C"]

plt.pie(sizes, labels=labels)
plt.show()
```

Uses

- Show proportions
- Display percentage distribution

Customizing Pie Charts

Example

```
plt.pie(
    sizes,
    labels=labels,
    autopct='%1.1f%%'
)
plt.title("Pie Chart")
plt.show()
```

Features

- Percentage display
- Labels
- Colors

Adding Legends

Example

```
plt.plot(x, y, label="Sales")
plt.legend()
plt.show()
```

Purpose

- Identify data series
- Improve readability

Multiple Plots

Example

```
plt.subplot(1, 2, 1)
plt.plot(x, y)

plt.subplot(1, 2, 2)
plt.bar(categories, values)

plt.show()
```

Benefits

- Compare different charts
- Better presentation

Importance of Data Visualization

Advantages

- Easy understanding of data
- Better decision making
- Identify trends and patterns
- Effective communication of results
